Day 53: Remove Product from Cart Method

Day 53: Remove Product from Cart Method

1. Day Number

Day 53

2. Topic Name

Method with Search and Remove

Today you will practice how an object method can:

- search through a list
- check a condition
- find a matching object
- remove that object from the list

---

3. Connection

Yesterday, you learned how to **add a Product object to a Cart** using an `add_product()` method and `append()`.

Conceptually, your cart now looks like:

```
Cart
 |
 +-- items
      |
      +-- Product("Laptop")
      +-- Product("Mouse")
      +-- Product("Keyboard")
```

Today you will do the opposite:

```
User wants to remove "Mouse"
        |
        v
Search cart items
        |
        v
Find Product whose name is "Mouse"
        |
        v
Remove that Product
```

This introduces an important pattern:

> **Search → Check → Remove**

---

4. Important Topics

Loop

A loop lets you examine every product inside the cart.

Example idea:

```
for product in products:
    print(product.name)
```

---

if condition

You need to compare the product name with the name the user wants to remove.

Conceptually:

```
if product.name == target_name:
```

---

remove()

Python's list `remove()` method removes a specific item from a list.

Simple example:

```
numbers = [10, 20, 30]

numbers.remove(20)

print(numbers)
```

Output:

```
[10, 30]
```

With objects, you can also remove the actual object stored in the list.

Method

The removal logic belongs inside the `Cart` class because removing something from a cart is a **Cart behavior**.

Conceptually:

```
Cart
 |
 +-- items
 |
 +-- add_product()
 |
 +-- remove_product()
```

---

5. Foundational Notes

Objects can be stored inside lists

Your `Cart.items` list does not need to contain simple strings.

It can contain `Product` objects.

For example:

```
items
 |
 +-- Product object
 |     name = "Laptop"
 |
 +-- Product object
 |     name = "Mouse"
 |
 +-- Product object
       name = "Keyboard"
```

Therefore, when looping through the cart:

```
for product in cart_items:
```

`product` represents one `Product` object.

You can access its attributes:

```
product.name
product.price
product.stock
```

Search using an object's attribute

Suppose the user enters:

```
Mouse
```

You should not compare the entire Product object directly with `"Mouse"`.

Instead, compare its `name` attribute:

```
product.name
      |
      v
   "Mouse"
```

Then compare:

```
"Mouse" == "Mouse"
```

Remove the object, not only its name

Suppose the cart contains:

```
Product("Laptop")
Product("Mouse")
Product("Keyboard")
```

After finding the `Mouse` Product object, remove that actual object from the list.

Result:

```
Product("Laptop")
Product("Keyboard")
```

You should handle "not found"

A user could enter:

```
Phone
```

while the cart contains only:

```
Laptop
Mouse
Keyboard
```

Your program should not crash.

It can display something like:

```
Product not found
```

---

6. Easy Example

Let's understand the same idea using simple strings first.

Suppose:

```
fruits = ["apple", "banana", "mango"]

target = "banana"
```

The thinking is:

```
Check apple
apple == banana? No

Check banana
banana == banana? Yes

Remove banana
```

Final list:

```
["apple", "mango"]
```

Your cart problem uses exactly the same idea, except instead of strings you have **Product objects**.

Conceptually:

```
for every product in cart:
    compare product.name with requested name
```

---

7. Problem Statement

You already have:

- a `Product` class
- a `Cart` class
- an `items` list
- an `add_product()` method

Now add a:

```
remove_product()
```

method to the `Cart` class.

The method should:

1. Ask for or receive the product name.
2. Search through the cart's `items` list.
3. Compare each Product object's `name`.
4. If a matching product is found, remove it from the cart.
5. If no matching product exists, show an appropriate message.

Example cart:

```
Laptop
Mouse
Keyboard
```

User wants to remove:

```
Mouse
```

After removal:

```
Laptop
Keyboard
```

---

8. Concepts Used

You will use:

- class
- object
- method
- `self`
- object attributes
- list
- `for` loop
- `if` condition
- comparison using `==`
- `remove()`
- search logic

The important new combination is:

```
Method
  +
Loop
  +
Object attribute
  +
Condition
  +
List removal
```

---

9. Thought Process

Before writing code, think through the problem.

Step 1: Where are products stored?

Inside:

```
cart.items
```

Step 2: What information does the user provide?

A product name.

Example:

```
Mouse
```

Step 3: How do I inspect products?

Loop through the cart.

Conceptually:

```
for each product in cart items
```

Step 4: How do I know whether it is the correct product?

Check:

```
product.name == requested_name
```

Step 5: What happens when the product matches?

Remove that Product object from the list.

Step 6: Should the search continue?

For this beginner problem, once one matching product has been removed, you normally do not need to keep searching.

Think about how you can stop the loop after removal.

Step 7: What if nothing matches?

Display something like:

```
Product not found
```

The overall logic is:

```
Product name
     |
     v
Loop through cart
     |
     v
Does product.name match?
     |
   +----+
   |    |
  Yes   No
   |    |
   v    v
Remove Continue searching
   |
   v
Stop
```

---

10. Pseudocode

```
CREATE remove_product method

    RECEIVE or ASK for product name

    FOR each product inside cart items

        IF product name matches requested name

            REMOVE that product from cart items

            DISPLAY removal message

            STOP searching

    IF no matching product was found

        DISPLAY "Product not found"
```

Notice that the pseudocode does **not** tell you the exact Python implementation.

That part is your exercise.

---

11. Suggested Solving Approach

OOP-Based Approach

The removal behavior should belong to `Cart`.

Your design can look conceptually like:

```
Product
 |
 +-- name
 +-- price
 +-- stock


Cart
 |
 +-- items
 |
 +-- add_product(product)
 |
 +-- remove_product(product_name)
```

Usage conceptually:

```
Create Product objects
        |
        v
Create Cart object
```

```
        |
        v
Add Product objects
        |
        v
Call remove_product()
        |
        v
Cart searches its own items
        |
        v
Matching Product removed
```

This is better than putting all the cart-removal logic outside the class because the `Cart` object should manage its own contents.

---

12. Easy Edge Cases

Edge Case 1: Product not found

Cart:

```
Laptop
Mouse
```

User enters:

```
Keyboard
```

Expected behavior:

```
Product not found
```

Cart should remain unchanged.

```
Laptop
Mouse
```

Edge Case 2: Empty cart

Cart:

```
[]
```

User enters:

```
Mouse
```

There are no Product objects to search.

Expected behavior could be:

```
Product not found
```

or:

```
Cart is empty
```

For today's exercise, either simple behavior is acceptable unless you specifically decide otherwise.

Edge Case 3: Product exists

Cart:

```
Laptop
Mouse
Keyboard
```

User enters:

```
Mouse
```

Expected cart:

```
Laptop
Keyboard
```

---

13. Expected Input

Assume the cart already contains products such as:

```
Laptop
Mouse
Keyboard
```

Then the user enters:

```
Mouse
```

You can think of the input as:

```
Enter product name to remove: Mouse
```

---

14. Expected Output

If the product exists:

```
Product removed: Mouse
```

The cart might then contain:

```
Laptop
Keyboard
```

If the product does not exist:

```
Product not found
```

For an empty cart:

```
Product not found
```

or a simple message such as:

```
Cart is empty
```

---

15. Hint Only

Inside your `Cart` method, think about this structure:

```
for product in self.items:
    if __________ == __________:
        self.items.remove(__________)
        # stop searching
```

The three questions to solve are:

1. **Which attribute of `product` should you compare?**
2. **What should it be compared with?**
3. **What should you pass to `remove()` — the product's name or the Product object itself?**

Remember:

```
self.items
   |
   +-- Product object
   +-- Product object
   +-- Product object
```

So you are searching for the **Product object whose `name` matches the requested name**, then removing that object.

Day 53 key idea:

```
Search → Match product.name → Remove Product object
```